Assignment 4: Optimizing the Traveling Salesman Problem

Problem Context:

The Traveling Salesman Problem is an optimization problem in which a set of cities must be visited once while minimizing the total travel distance. Since the number of possible tours grows at a high rate with the number of cities, brute force style approaches quickly become unfeasible for moderate to larger problem sizes. This project implements Simulated Annealing and Genetic Algorithm to improve a tour produced by a Nearest Neighbor heuristic.
Both algorithms explore the space of possible permutations in order to minimize the tour length. The goal of the experiment is to compare the convergence of SA and GA, analyze variability of the search, and determine whether SA escapes local minima and whether GA loses population diversity

Implemented Solutions:

Simulated Annealing: Simulated Annealing is inspired by the annealing process in metallurgy where a material is heated to a very high temperature then slowly cooled to reach a low energy state. The algorithm probabilistically accepts worse solutions early in the search to escape local minima.

Acceptance Rule for SA:
If a candidate tour is better: $\Delta E = cand_len - cur_len \leq 0$ the move is always accepted.

Otherwise it is accepted with probability $P = e^{-\Delta E/T}$ where T is the current temperature.

Respective Code Snippet:

```
cand_tour = random_two_opt_neighbor(cur_tour, rng)
cand_len = tour_length(cities, cand_tour)
delta = cand_len - cur_len

accept = False
# TODO: acceptance rule
# Is accept True or false when delta <= 0 or delta >0

# Acceptance rule:
# If better or equal, always accept.
# Otherwise accept with probability exp(-delta / T)
if delta <= 0:
    accept = True
else:
    accept = rng.random() < math.exp(-delta / T)
```

Genetic Algorithm: The Genetic Algorithm evolves a population of tours through selection, crossover, and mutation.
Each generation consists of a Tournament Selection, an Order Crossover, and a Mutation

Tournament Selection:
K random candidates are chosen from the population, the index of shortest length among the candidates is stored and returned

```
def tournament_select(pop: List[Tour], lengths: List[float], k: int, rng: random.Random) -> Tour:
    # Pick k random individuals and return the one with smallest tour length.
    candidates = rng.sample(range(len(pop)), k)
    winner_idx = min(candidates, key=lambda i: lengths[i])
    return pop[winner_idx][:]
```

Order Crossover:
Order crossover copies a segment from one parent while maintaining city ordering from the second parent.

```
# Copy sampled segment from parent1, +1 to include right bound
child[i:j + 1] = parent1[i:j + 1]

# Fill remaining slots using parent2 order, starting after j and wrapping around if necessary
p2_idx = (j + 1) % n
child_idx = (j + 1) % n
```

Mutation:
Mutation is implemented using a 2-opt swap, which reverses a segment of the tour.

```
#mutate swap using two_opt_swap
def mutate_swap(tour: Tour, rng: random.Random) -> Tour:
    n = len(tour)
    i, j = sorted(rng.sample(range(n), 2))
    return two_opt_swap(tour, i, j)
```

Experimental Results

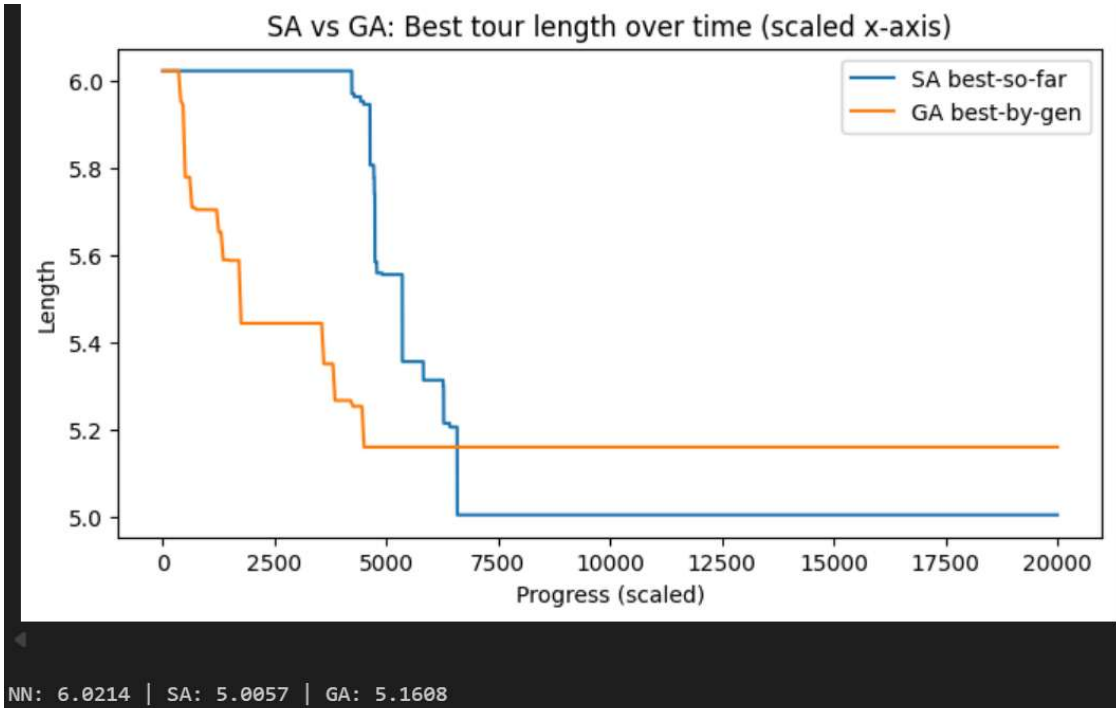
Final Results:

Algorithm	Tour Length	Improvement
NN	6.0214	baseline

SA	5.0057	16.9%
GA	5.1608	14.3%

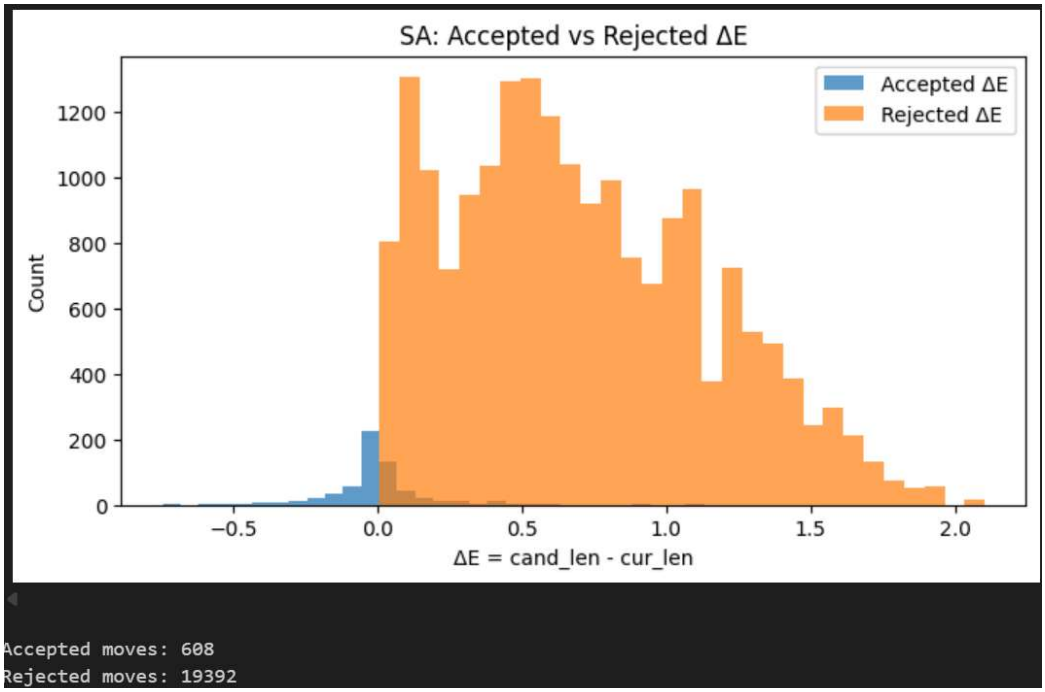
Both algorithms significantly improved the NN solution.

Convergence Behavior



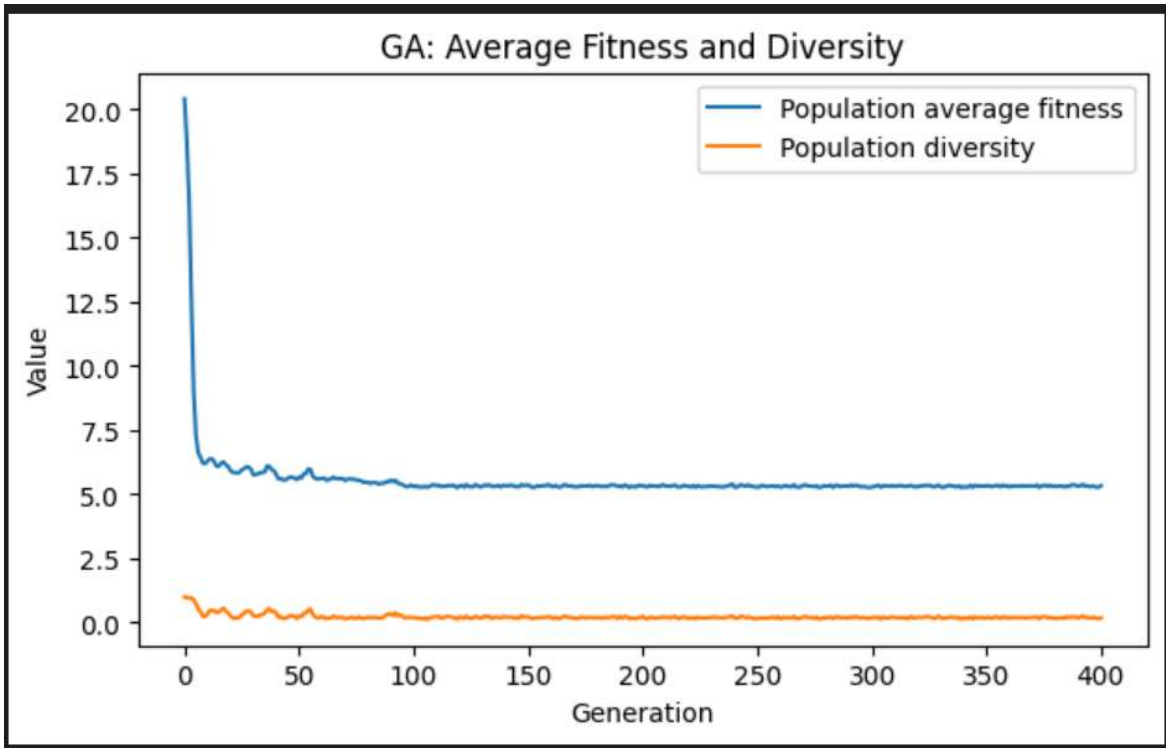
The convergence plot shows the best tour length over time.
GA converges faster initially, improving quickly during early generations, however SA continues improving for longer, eventually finding a better final solution.

Simulated Annealing ΔE Histogram



The ΔE histogram shows the distribution of accepted and rejected moves. Many positive ΔE values were rejected. Some positive ΔE values were accepted, which confirms that SA sometimes accepts worse solutions, and negative ΔE values dominated accepted moves.

GA Population Statistics



The GA instrumentation tracked population average fitness and population diversity. Average fitness improves rapidly early in the run, while population diversity decreases significantly(1.0 is high diversity) as generations progress. This demonstrates convergence toward similar solutions.

Analysis of Experimental Results

<u>Which algorithm converged faster?</u>
The Genetic Algorithm converged faster initially, showing rapid improvement in early generations, however, Simulated Annealing continued improving for a longer period and ultimately produced a better final solution.

<u>Which showed more variability?</u>
Simulated Annealing showed greater variability. The ΔE histogram shows both negative and positive ΔE values. This indicates the algorithm explored both improving and worsening moves. GA shows less variability because pressure from selection pushes the population toward better solutions.

<u>Did Simulated Annealing escape local minima?</u>
Yes, the ΔE histogram shows accepted positive ΔE values, meaning worse solutions were occasionally accepted allowing the algorithm to escape local minima and explore new regions of the search space.

Did the Genetic Algorithm lose diversity?
Yes, the population diversity curve decreases significantly over generations. This indicates that the population becomes dominated by similar tours. This loss of diversity is expected when a GA converges toward a optimal solution.

Conclusion

Both algorithms significantly improved the initial Nearest Neighbor tour. GA converged faster initially, SA produced the best final solution, SA showed greater search variability, SA escaped local minima, and GA gradually lost population diversity as it converged. Overall, Simulated Annealing demonstrated stronger final optimization in this experiment, while the Genetic Algorithm provided faster convergence early on.